



Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 6

Belo Horizonte
25 de junho de 2026

SUMÁRIO

1	PARTE 1	3
1.1	Exercício 1	3
1.2	Exercício 2	3
1.3	Exercício 3	4
1.4	Exercício 4	5
1.5	Exercício 5	5
1.6	Exercício 6	6
 ANEXO A CÓDIGO EXERCÍCIO 9.2		 9

1 PARTE 1

1.1 Exercício 1

Considere uma lei de conservação genérica dada por

$$\frac{\partial u}{\partial t} + \frac{\partial f}{\partial x} = 0$$

Aproximamos u e f por u_h e f_h , o que resulta em um Resíduo R dado por

$$R = \frac{\partial u_h}{\partial t} + \frac{\partial f_h}{\partial x}$$

Usando a técnica dos resíduos ponderados, temos que em cada elemento k

$$\int_{D^k} R \cdot v \, dx = 0$$

Fazendo v como a função de Lagrange e substituindo a expressão de R , temos

$$\int_{D^k} \left(\frac{\partial u_h}{\partial t} + \frac{\partial f_h}{\partial x} \right) L_j \, dx = 0$$

$$\int_{D^k} \frac{\partial u_h}{\partial t} L_j \, dx + \int_{D^k} \frac{\partial f_h}{\partial x} L_j \, dx = 0$$

Aplica integral por partes na segunda parcela da soma

$$\int_{D^k} \frac{\partial u_h}{\partial t} L_j \, dx = \int_{D^k} \frac{dL_j}{dx} f \, dx - [L_j f]_{x_L}^{x_R} = 0$$

Justificar segunda integral por partes, completar

1.2 Exercício 2

Dentro de um elemento D^k , aproximamos $u(x, t)$ por uma interpolação por polinômios de Lagrange (nodal)

$$u(x, t) = u_h(x, t) = \sum_{i=1}^{N_p} u_i(t) L_i(x)$$

Logo,

$$\frac{\partial u_h}{\partial t} = \sum_{i=1}^{N_p} \frac{du_i}{dt} L_i$$

Substitui isso na forma fraca

$$\int_{D^k} \sum_{i=1}^{N_p} \frac{du_i}{dt} L_i L_j dx = \int_{D^k} \frac{dL_j}{dx} f dx - [L_j f]_{x_L}^{x_R} = 0$$

$$\sum_{i=1}^{N_p} \frac{du_i}{dt} \int_{D^k} L_i L_j dx = \int_{D^k} \frac{dL_j}{dx} f dx - [L_j f]_{x_L}^{x_R} = 0$$

Definindo

$$M_{ij} = \int_{D^k} L_i L_j dx$$

Temos

$$M \frac{du}{dt} = \dots$$

Agora analisamos a segunda integral da forma fraca. Substituindo $f = au$, temos

$$\int_{D^k} \frac{dL_j}{dx} au dx$$

Substituindo u_h ,

$$\int_{D^k} \frac{dL_j}{dx} a \sum_{i=1}^{N_p} u_i(t) L_i(x) dx$$

$$a \sum_{i=1}^{N_p} u_i(t) \int_{D^k} \frac{dL_j}{dx} L_i(x) dx$$

Definindo

$$S_{ij} = \int_{D^k} \frac{dL_j}{dx} L_i(x) dx$$

Temos

$$M \frac{d\mathbf{u}}{dt} = aS\mathbf{u} - \mathbf{F}$$

Usar a representação nodal é mais conveniente do ponto de vista de código uma vez que armazenamos os pontos (nós) em que a função u_h é avaliada.

1.3 Exercício 3

Quando $\alpha = 1$, a segunda parcela se anula e temos apenas a média aritmética das bordas de ambos elementos vizinhos.

Quando $\alpha = 0$, o fluxo leva em consideração a direção de propagação por meio dos termos $\hat{n}^+$ e $\hat{n}^-$.

1.4 Exercício 4

Os pontos GLL são escolhidos ao invés dos pontos equidistantes pois ele apresentam melhor desempenho no método numérico. Além disso, a matriz M se torna aproximadamente diagonal ao escolher os pontos GLL, o que melhora o desempenho computacional.

Os pontos GLL são tabelados, tornando fácil seu uso.

1.5 Exercício 5

O método RK4 é alta ordem, sendo natural seu uso com o DGTD que também é de alta ordem. Usar o método de Euler após o obter a forma semi-discreta iria jogar fora todo o ganho de acurácia que obtemos com o DGTD.

Na forma semi-discreta, temos a forma

$$M \frac{d\mathbf{u}}{dt} = aS\mathbf{u} - \mathbf{F}$$

$$\frac{d\mathbf{u}}{dt} = (M)^{-1}aS\mathbf{u} - \mathbf{F}$$

$$\frac{d\mathbf{u}}{dt} = R(\mathbf{u})$$

que pode ser resolvido através de

$$\frac{d\mathbf{u}}{dt} = R(\mathbf{u})$$

$$\mathbf{k}_1 = R(\mathbf{u}^n)$$

$$\mathbf{k}_2 = R\left(\mathbf{u}^n + \frac{\Delta t}{2}\mathbf{k}_1\right)$$

$$\mathbf{k}_3 = R\left(\mathbf{u}^n + \frac{\Delta t}{2}\mathbf{k}_2\right)$$

$$\mathbf{k}_4 = R(\mathbf{u}^n + \Delta t \mathbf{k}_3)$$

$$\mathbf{u}^{n+1} = \mathbf{u}^n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

Mostra pq usa low storage RK4 em vez do RK4 convencional, qual a diferença

1.6 Exercício 6

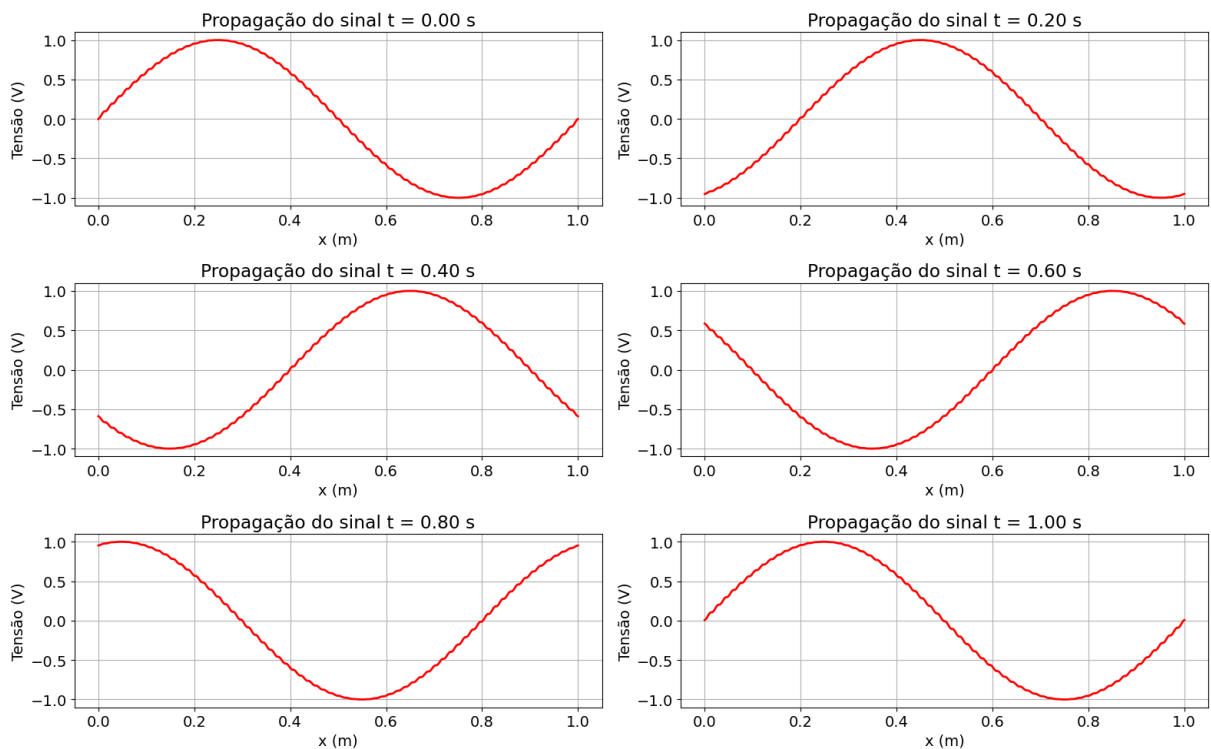
Todo o código usado nesse exercício encontra-se no Anexo ??.

Usando o método do DGTD para a equação da advecção 1D e os seguintes parâmetros de discretização:

- $K = 64$
- $N = 4$
- $\Delta t = 1.25 \cdot 10^{-4} \text{ s}$ (CFL = 0.2)
- $\Delta x = 1.5 \cdot 10^{-2} \text{ m}$
- $T = 1 \text{ s} \implies N_t = 8000$

podemos simular um período completo de uma advecção 1D com velocidade $a = 1$. O resultado obtido encontra-se na Figura 1. Vemos que o sinal se propaga para a direita e retorna para a posição inicial após um período completo, com a condição de contorno periódica.

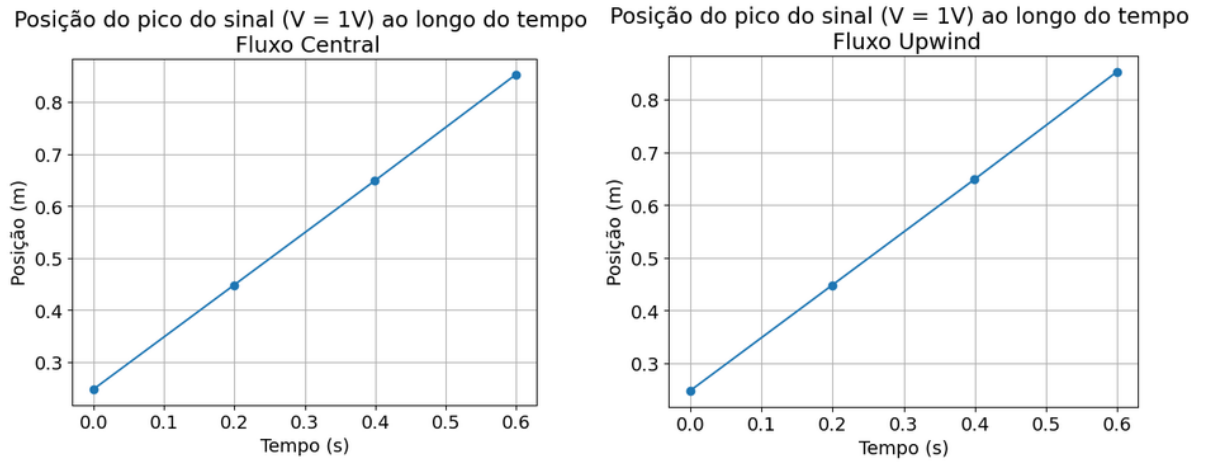
Figura 1 – Propagação da advecção 1D ao longo do tempo.



Monitorando a posição valor máximo do ponto ao longo do tempo, podemos traçar sua posição ao longo do tempo e assim calcular a velocidade de propagação numérica para os fluxos central e upwind exibido na Figura 2.

Em ambos casos, a inclinação da reta na Figura 2 nos retorna a velocidade de propagação numérica e, a partir de uma regressão linear simples, obtemos $a_{NUM} =$

Figura 2 – Posição do pico do sinal ao longo do tempo para ambos os fluxos.



1.009 m/s, com um erro de apenas 0.9 % em relação ao valor analítico, sendo esse um indicador do funcionamento da implementação.

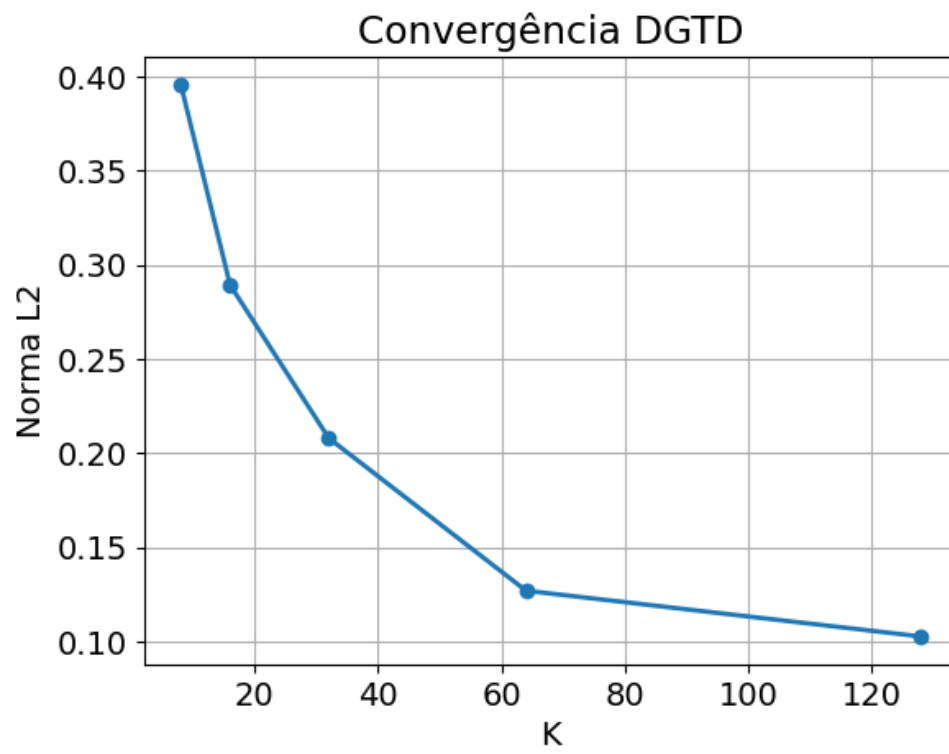
Uma outra maneira de testar a implementação é comparar com a solução analítica, dada por

$$u_{exact}(x, t) = u_0(x - at) = \sin(2\pi(x - t))$$

através da norma- L^2 e analisar a convergência conforme aumentamos o número de elementos K no domínio. O resultado da convergência está exibido na Figura 3.

Era esperado que a norma se reduzisse por um fator mínimo de 2 a cada vez que K dobra. No entanto, isso não é observada na Figura 3, indicando que há possibilidades de melhorar o código.

Figura 3 – Convergência da norma-L2 conforme aumenta-se o número de elementos K .



ANEXO A – CÓDIGO EXERCÍCIO 9.2

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def compute_sigma(N, ds, n_pml, sigma_max, m_order):
    sig = np.zeros(N)
    for i in range(N):
        dist_left = (n_pml - i) * ds
        dist_right = (i - (N - 1 - n_pml)) * ds
        if dist_left > 0:
            sig[i] = sigma_max * (dist_left / (n_pml * ds))**m_order
        elif dist_right > 0:
            sig[i] = sigma_max * (dist_right / (n_pml * ds))**m_order
    return sig

# --- parâmetros físicos ---
eps = 8.854187817e-12 # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7 # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
Nx = Ny = 200
dx = dy = 1e-3
dt = 0.9 / (vp * np.sqrt(1/dx**2 + 1/dy**2))
Nt = 800
L = Nx * dx
T = Nt * dt

# --- PML ----
npml = 30
m = 3
R_err = 1e-6
eta = np.sqrt(mu / eps)
d_pml = npml * dx
sig_max = -(m + 1) * np.log(R_err) / (2 * eta * d_pml)

# Declara os campos
Ez = np.zeros((Nx, Ny))
Hx = np.zeros((Nx, Ny))
Hy = np.zeros((Nx, Ny))

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_t_Ez = np.linspace(0, T, Nt + 1)

# CPML
psi_hxy = np.zeros((Nx, Ny - 1))
psi_hyx = np.zeros((Nx - 1, Ny))
psi_ezx = np.zeros((Nx - 2, Ny - 2))
psi_ezy = np.zeros((Nx - 2, Ny - 2))

sig_x_1D = compute_sigma(Nx, dx, npml, sig_max, m)
sig_y_1D = compute_sigma(Ny, dy, npml, sig_max, m)
sig_x, sig_y = np.meshgrid(sig_x_1D, sig_y_1D, indexing='ij')

b_x = np.exp(-sig_x * dt / eps)
c_x = b_x - 1.0
b_y = np.exp(-sig_y * dt / eps)

```

```

c_y = b_y - 1.0

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(100, 240, 6, dtype=int)
Ez_snapshots = []

# ponta de prova
px, py = Nx // 2 + 50, Ny // 2 + 0
probe = []

for n in range(Nt):
    # atualiza Hx
    dEz_y = (Ez[:, 1:] - Ez[:, :-1]) / dy
    psi_hxy = b_y[:, :-1] * psi_hxy + c_y[:, :-1] * dEz_y
    Hx[:, :-1] -= (dt / mu) * (dEz_y + psi_hxy)

    # atualiza Hy
    dEz_x = (Ez[1:, :] - Ez[:-1, :]) / dx
    psi_hyx = b_x[:-1, :] * psi_hyx + c_x[:-1, :] * dEz_x
    Hy[:-1, :] += (dt / mu) * (dEz_x + psi_hyx)

    # atualiza Ez
    dHy_x = (Hy[1:-1, 1:-1] - Hy[:-2, 1:-1]) / dx
    dHx_y = (Hx[1:-1, 1:-1] - Hx[1:-1, :-2]) / dy
    psi_ezx = b_x[1:-1, 1:-1] * psi_ezx + c_x[1:-1, 1:-1] * dHy_x
    psi_ezy = b_y[1:-1, 1:-1] * psi_ezy + c_y[1:-1, 1:-1] * dHx_y
    Ez[1:-1, 1:-1] += (dt / eps) * ((dHy_x + psi_ezx) - (dHx_y + psi_ezy))

    # fonte
    Ez[Nx//2, Ny//2] += np.exp(-0.5 * ((n * dt - (40 * dt)) / (12 * dt))**2)

    # ponta de prova
    probe.append(Ez[px, py])

    # salva para os snapshots
    if n in n_snapshots:
        Ez_snapshots.append(Ez.copy())

```